

On Experimenting Refactoring Tools to Remove Code Smells

Francesca Arcelli
Fontana
University of Milano-Bicocca
Milan, Italy
arcelli@disco.unimib.it

Domenico Pochiero
University of Milano-Bicocca
Milan, Italy
d.pochiero@
campus.unimib.it

Marco Mangiacavalli
University of Milano-Bicocca
Milan, Italy
m.mangiacavalli@
campus.unimib.it

Marco Zanoni
University of Milano-Bicocca
Milan, Italy
zanoni@disco.unimib.it

ABSTRACT

When we develop a software project of a certain complexity, source code maintainability could become a problem, in particular if developers do not use a consolidate development process that simplifies the management of the entire project. When source code becomes very complex, it is difficult for developers to share and modify it. We can improve internal software qualities such as reusability, maintainability and readability through refactoring. Refactoring can be applied to remove possible problems in the code, as code smells. Identifying code smells and removing them through refactoring results in better code maintainability, but it can be an overwhelming task. In this paper, we describe our experimentation on using four refactoring tools to remove code smells in four systems, with the aim to outline advantages and disadvantages of the tools with respect to the accomplishment of this task, and to identify the smells easier to be removed among the ones we considered in this paper.

CCS Concepts

•Software and its engineering → Software maintenance tools; Maintaining software; *Software evolution*;

Keywords

Code smells, refactoring

1. INTRODUCTION

Source code maintainability is the most expensive activity to deal with during the development and evolution of an application. This is due to many reasons, because changes inevitably occur in a software to remain useful, updated and

with high quality. Through refactoring we improve existing code, preserving its behavior. Fowler proposed many kinds of refactoring that can be used to remove code smells from the code [6]. Code smells are symptoms of problems at the code or design level, but they do not always represent problems (and have to be removed): they only give us hints on possible parts to be better inspected. When they are identified as critical, it is better to remove them as soon as possible through refactoring. However, some studies assert that developers often are reluctant to refactor source code, so the longevity of some smells increases during software evolution [5]. Refactoring can be expensive in terms of time and many tools have been developed to support a large number of different refactoring steps.

In this paper, we focus our attention on four refactoring tools than can be used to remove code smells. Some tools provide support also for the detection of code smells. We experimented these tools, by removing different kinds of smells, previously detected with well known code smell detection tools. After the refactoring of each smell, we checked if all the smells have been successfully removed and no other smell has been introduced. We are not interested in comparing the results of code smell detection tools. They often provide different results, due to different detection rules and metric thresholds used for smell detection [1, 9].

The aim of this paper is to provide answers to the following questions:

- *Question 1*: which refactoring tool results more useful in removing smells according to our experimentation?
- *Question 2*: which smell is easier to be removed using a given set of refactoring tools?
- *Question 3*: given a specific smell, which tool is more useful to remove it?

To answer the three above questions, we provide an initial framework to compare and evaluate the refactoring tools. We propose to consider different features, that can be relevant to evaluate the tools. Obviously, some features are objective, as for example the number of refactoring supported by a tool, while the evaluation of other features is subjective. In this case, the evaluation has been done by several

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

XP 2015 Workshops May 25–29, 2015, Helsinki, Finland

© 2015 ACM. ISBN 978-1-4503-3409-9/15/05...\$15.00

DOI: <http://dx.doi.org/10.1145/2764979.2764986>

students of a Master course of reverse engineering at our University. We started by experimenting four refactoring tools on four systems of the Qualitas Corpus [17].

We hope, through our experimentation, to provide some insights to developers for the choice of the refactoring tool to be used to remove smells and also some suggestions for those involved in the development or improvement of existing refactoring tools.

The paper is organized through the following sections: in Section 2 we introduce some related work, in Section 3 we briefly describe the refactoring tools we experimented; in Section 4 we introduce the code smell detection tools we used, in Section 5 we give answers to the three research questions and describe the features we used to evaluate the tools, in Section 6 we outline some threats to validity of our work and finally in Section 7 we conclude and outline some future developments.

2. RELATED WORK

With respect to refactoring and code smells, we can find in the literature many works in several directions, e.g., works on suggesting the right refactoring to remove a smell by exploiting some kind of visualization [14], works on the investigation of the most critical smells by evaluating the impact on software quality metrics before and after the refactoring [3], the Refactoring Workbook of Wake [18] on the identification of smells and their removal, the work of Counsell et al. [7] that established a link between refactoring and code smells in terms of fan in and fan out-degrees on a dependency graph. Another interesting and recent study on refactoring has been proposed by Szoke et al. [16], where they provide answers to different questions on refactoring best practices of large scale industrial systems. A very interesting and complete survey on software refactoring in general has been proposed by Mens et al. [12], and relevant considerations on how refactoring is done have been made by Murphy-Hill [13].

With respect to experience reports on using refactoring tools, we found the MSc thesis of Drosdz [8], where the author proposed a critical analysis of two refactoring tools, and the work of Simmond et al. [15], where the authors compare some refactoring tools, but the emphasis is not on the effectiveness of the tools in removing code smells. In this paper, we consider four refactoring tools and we focus our attention on the usefulness or drawbacks of these tools for smells refactoring.

3. REFACTORIZING TOOLS

Refactoring can be used in different contexts, to improve the design and code quality of existing systems. In this section, we briefly describe the tools that we have considered and used during our experimentation.

- *Eclipse*¹: well known open source IDE, it is primarily used to develop Java applications. It offers a customizable environment and many functionalities including a large set of refactoring functions.
- *IntelliJ IDEA*²: it is a commercial IDE developed by JetBrains. This software offers several functionalities to speed up the development process and improve the

quality of source code; the principal functions provided by IntelliJ IDEA are related to software analysis, refactoring support and detection of code smells.

- *JDeodorant*³: it is an Eclipse plug-in that can be used to identify and remove code smells. According to our knowledge, JDeodorant is the only software currently available able to provide code smell detection and then to suggest which refactoring to apply to remove on the detected smells.
- *RefactorIT*⁴: it is a commercial Eclipse and NetBeans plugin that allows to compute some metrics and apply different types of refactoring.

In Table 1, we outline some features about the refactoring tools we have considered.

Table 1: Refactoring tools

	Eclipse	IntelliJ IDEA (C.E.)	JDeodorant	RefactorIT
Version	4.2.0	13.0.2	5.0	2.7
Dimension	253MB	345MB	777KB	10 MB
Operating System	Windows, Mac, Linux	Windows, Mac, Linux	Windows, Mac, Linux	Windows, Mac, Linux
License	Free	Commercial	Free	Commercial
Supported languages	More lan- guages	Java, JavaScript, PHP, Ruby, Python, Scala	Java	Java

We started our ongoing experimentation using the above tools because Eclipse, IntelliJ IDEA and RefactorIT are well known tools and JDeodorant is able to detect smells and to suggest the refactoring steps. Moreover, we decided to experiment both commercial and free IDEs and tools. There are many other refactoring tools available, free or commercial, e.g., JRefactory⁵, the source code analyzer PMD, or IDEs like NetBeans or Oracle Java Developer, which we can consider in future experimentations.

4. CODE SMELL DETECTION

For our experimentation, we used for the detection of code smells the following tools:

- *iPlasma*⁶: it is a tool developed for the evaluation of the quality of object-oriented programs; it can be used to calculate different metrics and to identify the most critical points that can be improved through code smell detection and their removal. iPlasma allows to detect 11 code smells.
- *JDeodorant*: this software has already been presented in the previous section; it can be used to detect four types of smells: God Class, Long Method, Feature Envy and Type Checking.

³<http://www.jdeodorant.com/>

⁴<http://sourceforge.net/projects/refactorit/>

⁵<http://jrefactory.sourceforge.net/>

⁶<http://loose.upt.ro/reengineering/research/iplasma>

¹<https://www.eclipse.org/>

²<http://www.jetbrains.com/idea/>

- *InCode*⁷: it is a commercial tool developed by Intooitus for code smell detection in Java, C and C++ programs. It can be used in standalone mode or as an Eclipse plug-in. InCode allows to detect 10 code smells. InCode and InFusion (a larger tool from the same company) are commercial successors of iPlasma.

We decided to use JDeodorant because it supports both detection and refactoring, iPlasma because the tool is free and allows to detect a large number of smells, and InCode because it is able to detect the Data Clumps code smell. There are other tools that can be used for code smell detection, e.g., Décor, Checkstyle, CodeProAnalytics, JBuilder, and CodeVizard.

The definitions of the smells that we have considered are the following [6, 11]:

- *God Class*: class that performs too much work on its own, delegating only minor details to a set of trivial classes and using the data from other classes. This has a negative impact on the reusability and the understandability of that part of the system.
- *Brain Method*: method that tend to centralize the functionality of a class, in the same way as a God Class can centralize the functionality of an entire subsystem, or sometimes even a whole system.
- *Brain Class*: class that tends to accumulate an excessive amount of intelligence, usually in the form of several methods affected by Brain Method.
- *Data Clumps*: bunches of data that hang around together really ought to be made into their own object.
- *Feature Envy*: data and behavior that act on that data belong together. When a method makes too many calls to other classes to obtain data or functionality, Feature Envy can be present.
- *Intensive Coupling*: when a method is tied to many other operations in the system, whereby these provider operations are dispersed only into one or a few classes.
- *Long Method*: a long method in terms of line of code, that can be also complex. Systems that have a majority of small methods tend to be easier to extend and maintain because they are easier to understand and contain less duplication.
- *Shotgun Surgery*: this smell is evident when one must change lots of pieces of code in different places simply to add a new or extended piece of behavior.
- *Type Checking*: a complicated conditional expression to select a variant of an algorithm based on the value of an attribute.

In Table 2, we outline the tools used for the detection and the refactoring of the smells. As we can see from the table, we removed each smell by experimenting at least two refactoring tools and in some cases we used all the four tools. For what concerns the detection, we primarily used the tools InCode and iPlasma. In few cases we experimented more than one tool, i.e., for the Feature Envy, God Class and Type Checking smells.

⁷<http://www.intooitus.com/products/incode>

Table 2: Tools of detection and refactoring

	Detection	Refactoring
Brain Class	iPlasma	Eclipse, RefactorIT
Brain Method	iPlasma	Eclipse, RefactorIT
Data Clumps	InCode	Eclipse, IntelliJ IDEA, RefactorIT
Feature Envy	iPlasma, InCode, JDeodorant	Eclipse, IntelliJ IDEA, JDeodorant, RefactorIT
God Class	iPlasma, InCode, JDeodorant	Eclipse, JDeodorant RefactorIT
Intensive Coupling	iPlasma	Eclipse, RefactorIT
Long Method	JDeodorant	Eclipse, IntelliJ IDEA, JDeodorant, RefactorIT
Shotgun Surgery	iPlasma	Eclipse, IntelliJ IDEA
Type Checking	iPlasma, JDeodorant	JDeodorant, Eclipse

Table 3: Code smells and refactoring steps

	Refactoring steps
Brain Class	Extract Class and Move method
Brain Method	Extract Method
Data Clumps	Extract Class and Introduce Parameter Object
Feature Envy	Move method
God Class	Extract Class
Intensive Coupling	Move Method
Long Method	Extract method
Shotgun Surgery	Inline class
Type Checking	Replace Conditional with Polymorphism and Replace Type Code with State/Strategy

We experimented the tools on the following systems: AoI⁸, C-jdbc⁹, Cobertura¹⁰, Log4j¹¹. Some statistics are given in Table 4.

5. EXPERIMENTATION

We removed the smells through different refactoring tools (see Table 2) and different refactoring steps. In Table 3 we outline the refactoring steps that we applied to remove each smell. Then we tried to answer to the three questions posed in the Introduction. In Table 5 we show the number of detected smells in each system for each kind of smell and the number of smells that we have effectively removed experimenting the different refactoring tools. We can see that in some cases we removed all the found smells. In other cases, when the number of detected smells was high, or the refactoring of the smell could create more problems or involve the refactoring of several other smells, we removed only a significant subset of them.

After the refactoring of each smell, we checked its correct removal and if through the application of refactoring

⁸<http://www.artofillusion.org/>

⁹<http://c-jdbc.ow2.org/>

¹⁰<http://cobertura.github.io/cobertura/>

¹¹<http://logging.apache.org/log4j/2.x/>

Table 4: Analyzed systems

	Version	Classes	Packages	Methods	LOC
AoI	2.8.1	728	134	6,281	92,725
C-jdbc	2.0	737	179	409	80,415
Cobertura	1.4.9.1	135	25	456	99,745
Log4j	1.2.17	541	59	311	69,596

Table 5: Code smell detected and refactored

		BC	BM	DC	FE	GC	IC	LM	SS	TC
AoI	Detected	15	40	143	49	69	37	33	45	35
	Refactored	15	35	52	37	48	37	33	45	35
C-jdbc	Detected	25	12	46	144	62	18	9	31	34
	Refactored	25	12	20	20	20	18	9	31	34
Cobertura	Detected	7	232	27	24	37	20	87	35	19
	Refactored	7	45	27	24	37	20	40	25	19
Log4j	Detected	41	86	35	102	38	22	75	29	31
	Refactored	24	42	35	62	38	22	48	29	31

Legend BC = Brain Class, BM = Brain Method, DC = Data Clumps, FE = Feature Envy, GC = God Class, IC = Intensive Coupling, LM = Long Method, SS = Shotgun Surgery, TC = Type Checking

we introduced another kind of smell; both these two situations never occurred. During our experimentation we also used another tool called RefFinder [10] to verify the reliability of the applied refactorings, in particular in the case of automated refactoring. This Eclipse plugin can be used to compare different versions of a software and identify the applied refactorings. It is able to detect 65 out of the 72 refactorings reported in Fowler’s catalog [6].

5.1 An initial proposal of a framework to evaluate refactoring tools

We outline below some features about the refactoring tools, that we consider in our proposal of a framework for evaluating refactoring tools:

- *Code smell detection*: list of code smells identified by each tool.
- *Smell detection speed*: how fast the tool is in code smell detection.
- *Smell severity*: if the tool assigns to the detected code smells a level of criticality or intensity.
- *Supported refactoring*: number of refactorings provided by each tools.
- *Automated refactoring*: if the tool can remove code smells automatically.
- *Refactoring reliability*: how reliable is the refactoring applied by each tool according to the introduction of bugs or other problems.
- *Preview*: if the tool offers the possibility to visualize the source code before and after applying the refactoring.
- *Warnings*: if the tool provides some kind of warnings on the code.
- *Problems*: defects or problems encountered during the use of the tool.

- *Metrics*: if the tool support metrics computation.
- *Reports*: if the tool provides some kind of reports on the results obtained after refactoring.
- *Usability*: how intuitive and user friendly is the tool.
- *Scalability*: how the tool works when the size of the analyzed software increases.

Table 6: Features for evaluating refactoring tools

	Eclipse	IntelliJ IDEA (C.E.)	JDeodorant	RefactorIT
Code smell detection	No	Yes	No	No
Smell detection speed	-	M	L	-
Smell severity	No	Yes	No	No
Supported refactoring	27	35	5	23
Automated refactoring	No	No	Yes	No
Refactoring reliability	H	H	M	M
Preview	Yes	Yes	Yes	No
Warnings	Yes	Yes	Yes	No
Problems	No	Some crashes	Some crashes	Some crashes
Metrics	No	Yes	No	Yes
Reports	No	Yes	Yes	No
Usability	H	M	M	M
Scalability	H	M	L	H

In Table 6, we outline the above features for the considered refactoring tools. With the three labels H, M and L we denote respectively High, Medium and Low. For example, w.r.t. Usability, we found Eclipse very user friendly, IntelliJ IDEA intuitive but slow, JDeodorant slow in particular respect to smell detection and RefactorIT simple to use. Usability has been evaluated in particular according to the simplicity in using the tool for the first time, without prior knowledge of the procedure. We separately evaluated all the subjective features, like Usability, and then compared our personal outcomes. We always agreed in our evaluations, with no need for further discussion.

To complete the information regarding the detected code smells, we report in the following the smells supported by IntelliJ IDEA: Dead Code, Message Chain, Feature Envy, Long Parameter List, Long Method.

The following are instead the code smells detected by JDeodorant: Long Method, God Class, Feature Envy, Type Checking.

5.2 Answer to Question 1

The refactoring tool that we found more useful during our experimentation for removing smells is Eclipse. We found it very simple and intuitive to use, and through the supported

refactorings we were able to remove all the code smells we have analyzed. The code smell removal phase is very quick and never introduces new defects after refactoring. The tool allows to control the refactoring task, providing a preview of the software before the actual application of the refactoring step. The refactoring functionality is also highly customizable. For example, Extract Method provides parameters to customize the name of the extracted method, and the names and types of the parameters.

IntelliJ IDEA is a very powerful refactoring tool as well, and provides a high number of refactoring options. However, according to our experience, it is less customizable than Eclipse, offers less control on refactoring, and it is a commercial software. We experienced also some crashes while using it. RefactorIT is a tool very similar to Eclipse but we consider Eclipse better because RefactorIT offers less refactoring functionalities (in particular, Extract Class is not implemented), and it does not provide a preview of the refactoring to be applied.

The last tool in this rank is JDeodorant. It has not been developed as a refactoring tool, but as a tool combining both detection and refactoring. This makes it a very interesting software, as it provides completely automatic refactoring functions for four code smells. However, the automatic refactoring offered can sometimes introduce bugs or can deceive developers. Furthermore, during its execution, some crashes and shutdowns of the tool occurred. In any case, the idea to create a tool that can detect smells and suggest refactoring for their removal is very interesting, but it may introduce some risks which must not be underestimated. For example, automatic refactoring can lead developer to lose familiarity with the source code. Hence, the suggestions are very useful, but executing the entire process of refactoring automatically can be counter-productive.

5.3 Answer to Question 2

The code smells we found easier to remove given the considered refactoring tools are Feature Envy, Intensive Coupling and Type Checking. The first one is very simple, because it is sufficient to execute a Move Method in the smelly class and it is eliminated. We obtained very quick results through JDeodorant because it detects immediately in which class this method must be moved. In the second one, Intensive Coupling, the scenario is very similar: to eliminate it we have to apply a Move Method refactoring, but in this case we do not have support from JDeodorant. Finally, the Type Checking smell can be easily removed by applying “Replace Conditional with Polymorphism” or “Replace Type Code with State/Strategy”. Both the refactoring types are provided by Eclipse and JDeodorant. We found God Class and Brain Class the hardest smells to be removed, because classes affected by these code smells are long and more difficult to manage. The best tools which can be used to remove these code smells are certainly Eclipse and IntelliJ IDEA, for the high number of refactorings they provide.

5.4 Answer to Question 3

For every analyzed code smell, we outline, according to our experience, which tool is more useful in removing it and we make some observations about the other tools:

- *Brain Class*: the best tool to remove this smell was Eclipse.

- *Brain Method*: for this smell, very similar to the previous one, the best tool was Eclipse.
- *Data Clumps*: also in this case Eclipse because the extraction of the class is very simple; IntelliJ IDEA provides this kind of refactoring and it is very useful too; instead, RefactorIT is inferior because it does not support Extract Class refactoring.
- *Feature Envy*: the best software used to remove this code smell is JDeodorant; the automatic refactoring provided by this tool is very useful and never happened to insert defects, unlike for the refactoring of other kinds of smell; the other tools can be efficiently used to remove this smell, too.
- *God Class*: to remove this smell the best software results to be Eclipse, as it offers high control during the application of refactoring; we found also IntelliJ IDEA very useful, even if it offers less customization; RefactorIT does not provide an Extract Class refactoring, and this is very limiting; finally, JDeodorant often introduces some defects during the removal of this smell, but it is otherwise useful to suggest some guidelines for the execution of the right refactoring.
- *Intensive Coupling*: to remove this smell we consider RefactorIT the best software, due to its capacity of immediately individuate eventual conflicts which would be created applying refactoring. Eclipse and IntelliJ IDEA can also be used with success.
- *Long Method*: the best tools to remove this code smell are Eclipse and IntelliJ IDEA, because they provide very powerful Extract Method refactorings; RefactorIT is also a very good tool, but it does not provide a preview of the class before applying the refactoring and this represents an important drawback; instead, JDeodorant sometimes introduces errors after applying automatic refactoring.
- *Shotgun Surgery*: the best refactoring tool is Eclipse; we prefer it to IntelliJ IDEA for execution time, preview availability and usability.
- *Type Checking*: the best software is again Eclipse because it never introduced errors or any other kinds of problem during the refactoring. We tried also JDeodorant and we observed that it is less intuitive and immediate.

In general, we can observe that all these refactoring tools are very good to remove these kinds of smells, but IntelliJ IDEA and, in particular, Eclipse are the best for the high amount of refactoring options provided and their high usability.

5.5 Code smell refactoring examples

We show an example of code smell that can be eliminated by all the analyzed refactoring tools; the code smell has been detected and removed from the source code of the open source project Cobertura¹² version 1.4.9.1. The source code is available in the Appendix.

¹²<http://cobertura.github.io/cobertura/>

Listing 1 shows an example of Long Method. This method, called `_measureSource`, comes from class `Javancss` in package `net.sourceforge.cobertura.javancss` and it is 71 lines of code long.

Listing 2 shows the source code after applying two Extract Methods, refactoring the original method in three methods making the source code easier to maintain and understand.

The best tools to remove this code smell are Eclipse and IntelliJ IDEA that provide a powerful and customizable Extract Method; we consider RefactorIT a bit inferior because it does not provide a preview of the refactored code; instead, JDeodorant, after the execution of the Extract Method, sometimes inserts errors in the source code that must be manually resolved by the developers.

6. THREATS TO VALIDITY

For what concerns the threats to validity of our work, we have to outline that the experimentation has been done on four systems. We refactored in some cases all the smells we found, in other cases only a subset of the smells, but for each smell we tried to consider a number of cases meaningful enough to be able to experiment the tool and draw some considerations on its usefulness. Obviously, the experimentation will be extended on a larger set of systems and considering other tools or IDE supporting refactoring to remove smells.

7. CONCLUSION AND FUTURE DEVELOPMENTS

In this paper, we outlined different features of four refactoring tools and we described our experience on using them to remove code smells. We observed that only one tool, JDeodorant, is able to identify and suggest the right refactoring to remove code smells, but we encountered several problems in using this tool, both for detection and refactoring.

The refactoring options provided by Eclipse are very intuitive and complete and can largely speed up manually refactoring; the provided functions can remove all the analyzed smell kinds. RefactorIT also offers many refactoring functions, very similar to Eclipse, and can remove smells in a very simple and clean way. RefactorIT can also calculate metrics, on the contrary of Eclipse. Its limits are that it is not an open source tool and it cannot provide an Extract Class Method. IntelliJ IDEA is a very good software, offering a lot of functionalities for program analysis and refactoring options, very similar to Eclipse. We experienced crashes during its usage.

After this experimentation, we think that the main drawback of a refactoring tool is obviously the possibility to introduce defects or any other kinds of problem in the code after the refactoring. The ability of a tool to provide a refactoring preview is strictly linked to this aspect, and represents a way to ensure that the applied refactoring is not harmful. The main advantages of a tool to remove code smells are instead the availability of a large number of refactoring functionalities, the possibility to immediately identify the exact portion of code where to apply the refactoring, and the capability to provide some kind of refactoring suggestion. This last aspect is covered only by JDeodorant, which unfortunately can be used to remove only four smells. In conclusion, answers to Question 1 are useful to identify the appropriate tool to

be used to remove smells in general, without focusing on a specific smell.

Answers to Question 2 are useful to identify the smell on which we can focus first, because it is easier to remove it, instead of focusing on the most critical ones. One can decide to remove first the easiest to be removed and then spend more time on the most complex ones. The code smells we found easier to remove are Feature Envy, Intensive Coupling and Type Checking. The most difficult smells to remove have been God Class and Brain Class, because these classes can be very large and difficult to manage. Moreover, in some cases a smell can be correlated to other smells, and removing it implies to refactor also the other ones. It is important also to pay attention to the possible introduction of a new smell after some refactoring steps; for example, by removing a Shotgun Surgery we could introduce other smells, as Feature Envy.

Answers to Question 3 are associated to the answers of Question 1, but in some cases we can find a smell which is easier to remove with a specific tool respect to another one. We found Eclipse as the “winner” for Question 1, but for example, for Feature Envy smell we found more useful JDeodorant and for Intensive Coupling smell RefactorIT.

Our experimentation shows that a “perfect tool” for smell refactoring does not exist, even if we consider Eclipse very useful. In any case, there are several good softwares that, combined together, can offer very satisfying results. They can offer great support to developers, in particular in the contexts in which the source code of the analyzed system is very complex or written by other people.

In relation to our experimentation, according to the answers of the three questions, we evaluated through the framework the following features as the most useful: Number of Supported refactorings, Smell severity, Refactoring reliability and Preview.

In future developments, we aim to extend our experimentation and refine the framework, by experimenting other refactoring tools to provide a more deep and complete analysis. We can also take into account different categories of refactoring support, as IDE/tool, commercial/free tool and others. We aim to experiment for example JRefactory, CodeProAnalytics (for duplicate and dead code only), Borland Together, JCosmo or TRex, which offer different refactoring options. Moreover, we can consider other smells and provide new answers to Question 2 and 3 accordingly.

Certainly, the possibility to use the same tool to detect and refactor smells is very interesting and we hope that other tools in the future will take into account this aspect. It would be interesting also to offer not only a preview of the code and refactoring suggestions, but also some views showing possible correlations or co-occurrences among smells, that can be particularly useful during the refactoring process, also to locate the most critical portions of code, as we already started to investigate [2].

We did not consider in this paper the refactoring of duplicated code, since it is more difficult to decide which refactorings to apply to it in specific cases. We explored this aspect in recent work [4], by developing a tool suggesting the best refactoring to apply in case of duplicated code.

8. REFERENCES

- [1] F. Arcelli Fontana, P. Braione, and M. Zannoni. Automatic detection of bad smells in code: An

- experimental assessment. *Journal of Object Technology*, 11(2):5: 1–38, 2012.
- [2] F. Arcelli Fontana, V. Ferme, and M. Zanoni. Towards assessing software architecture quality by exploiting code smell relations. In *to appear in IEEE Proceedings of SAM 2015 workshop, co-located with ICSE 2015*, May 2015.
 - [3] F. Arcelli Fontana and S. Spinelli. Impact of refactoring on quality code evaluation. In *Proceedings of the 4th Workshop on Refactoring Tools (WRT '11)*, pages 37–40, Waikiki, Honolulu, HI, USA, May 2011. ACM.
 - [4] F. Arcelli Fontana, M. Zanoni, and F. Zanoni. A duplicated code refactoring advisor. In *Proceedings of the 16th International Conference on Agile Software Development (XP 2015)*. Springer, May 2015.
 - [5] R. Arcoverde, A. Garcia, and E. Figueiredo. Understanding the longevity of code smells: preliminary results of an explanatory survey. In *Proceedings of the 4th Workshop on Refactoring Tools (WRT '11)*, pages 33–36, Waikiki, Honolulu, HI, USA, May 2011. ACM.
 - [6] K. Beck, M. Fowler, and G. Beck. Bad smells in code. *Refactoring: Improving the design of existing code. Addison-Wesley Object Technology Series*, pages 75–88, 1999.
 - [7] S. Counsell, R. Hierons, H. Hamza, S. Black, and M. Durrand. Is a strategy for code smell assessment long overdue? In *Proceedings of the 2010 ICSE Workshop on Emerging Trends in Software Metrics (WETSoM '10)*, pages 32–38, Cape Town, South Africa, 2010. ACM.
 - [8] M. Z. Drozd. A critical analysis of two refactoring tools. 2008. Master Thesis. University of Pretoria.
 - [9] A. Hamid, M. Ilyas, M. Hummayun, and A. Nawaz. A comparative study on code smell detection tools. *International Journal of Advanced Science and Technology*, 2013.
 - [10] M. Kim, M. Gee, A. Loh, and N. Rachatasumrit. Ref-finder: a refactoring reconstruction tool based on logic query templates. In *Proceedings of the 18th ACM SIGSOFT international symposium on Foundations of software engineering (FSE '10)*, pages 371–372, Santa Fe, New Mexico, USA, Nov. 2010. ACM.
 - [11] M. Lanza, R. Marinescu, and S. Ducasse. *Object-oriented metrics in practice*. Springer, 2006.
 - [12] T. Mens and T. Tourwé. A survey of software refactoring. *IEEE Transactions on Software Engineering*, 30(2):126–139, 2004.
 - [13] E. Murphy-Hill, C. Parnin, and A. P. Black. How we refactor, and how we know it. *IEEE Transactions on Software Engineering*, 38(1):5–18, 2012.
 - [14] J. Pérez and Y. Crespo. Perspectives on automated correction of bad smells. In *Proceedings of the joint international and annual ERCIM workshops on Principles of software evolution (IWPSE) and software evolution (Evol) workshops (IWPSE-Evol '09)*, pages 99–108. ACM, Aug. 2009.
 - [15] J. Simmonds and T. Mens. A comparison of software refactoring tools. Technical report, 2002.
 - [16] G. Szőke, C. Nagy, R. Ferenc, and T. Gyimóthy. Case study of refactoring large-scale industrial systems to efficiently improve source code quality. In *Computational Science and Its Applications-ICCSA 2014*, pages 524–540. Springer, 2014.
 - [17] E. Tempero, C. Anslow, J. Dietrich, T. Han, J. Li, M. Lumpe, H. Melton, and J. Noble. Qualitas corpus: A curated collection of java code for empirical studies. In *2010 Asia Pacific Software Engineering Conference (APSEC2010)*, pages 336–345, Dec. 2010.
 - [18] W. C. Wake. *Refactoring workbook*. Addison-Wesley Professional, 2004.

APPENDIX

This section contains two examples (Listings 1 and 2) of code smells and their refactored versions. These examples have been discussed in Section 5.5.

```
private void _measureSource(Reader reader) throws
    IOException, Exception, Error {
    Util.debug("_measureSource(Reader).ENTER");
    Util.debug("_measureSource(Reader).ENTER2");
    try {
        boolean parser15 = _pInit != null && _pInit.getOptions()
            != null && _pInit.getOptions().get("parser15")
            != null;
        if (Util.isDebugEnabled()) {
            if (parser15) {
                Util.debug("creating _JavaParser15Debug");
                _pJavaParser = new JavaParser15Debug(reader);
            } else {
                Util.debug("creating _JavaParserDebug");
                _pJavaParser = new JavaParserDebug(reader);
            }
        } else {
            if (parser15) {
                Util.debug("creating _JavaParser15");
                _pJavaParser = new JavaParser15(reader);
            } else {
                Util.debug("creating _JavaParser");
                _pJavaParser = new JavaParser(reader);
            }
        }
        _pJavaParser.parse();
        Util.debug("Javancss._measureSource(DataInputStream).
            SUCCESSFULLY_PARSED");

        _ncss += _pJavaParser.getNcss();
        _loc += _pJavaParser.getLOC();
        _vFunctionMetrics.addAll(_pJavaParser.getFunction());
        _vObjectMetrics.addAll(_pJavaParser.getObject());
        Map htNewPackages = _pJavaParser.getPackage();
        for (Iterator ePackages = htNewPackages.entrySet().
            iterator(); ePackages.hasNext();) {
            String sPackage = (String) ((Map.Entry) ePackages.
                next()).getKey();

            PackageMetric pckmNext = (PackageMetric)
                htNewPackages.get(sPackage);
            pckmNext.name = sPackage;

            PackageMetric pckmPrevious = (PackageMetric)
                _htPackages.get(sPackage);
            pckmNext.add(pckmPrevious);

            _htPackages.put(sPackage, pckmNext);
        }
    } catch (Exception pParseException) {
        if (_sErrorMessage == null) {
            _sErrorMessage = "";
        }
        _sErrorMessage += "ParseException_in_STDIN";
        if (_pJavaParser != null) {
            _sErrorMessage += "\nLast_useful_checkpoint:_\" +
                _pJavaParser.getLastFunction() + "\"\n";
        }
        _sErrorMessage += pParseException.getMessage() + "\n";
        _throwError = pParseException;
        throw pParseException;
    } catch (Error pTokenMgrError) {
        if (_sErrorMessage == null) {
            _sErrorMessage = "";
        }
        _sErrorMessage += "TokenMgrError_in_STDIN\n";
        _sErrorMessage += pTokenMgrError.getMessage() + "\n";
        _throwError = pTokenMgrError;
        throw pTokenMgrError;
    }
}
```

Listing 1: Long Method example (original)

```
private void _measureSource(Reader reader) throws
    IOException, Exception, Error {
    Util.debug("_measureSource(Reader).ENTER");
    Util.debug("_measureSource(Reader).ENTER2");
    try {
        boolean parser15 = _pInit != null && _pInit.getOptions()
            != null && _pInit.getOptions().get("parser15") != null;
        && _pInit.getOptions().get("parser15") != null;
        manageDebugger(reader, parser15);
        _pJavaParser.parse();
        Util.debug("Javancss._measureSource(DataInputStream).
            SUCCESSFULLY_PARSED");

        _ncss += _pJavaParser.getNcss();
        _loc += _pJavaParser.getLOC();
        _vFunctionMetrics.addAll(_pJavaParser.getFunction());
        _vObjectMetrics.addAll(_pJavaParser.getObject());
        iteratePkgs();
    } catch (Exception pParseException) {
        if (_sErrorMessage == null) {
            _sErrorMessage = "";
        }
        _sErrorMessage += "ParseException_in_STDIN";
        if (_pJavaParser != null) {
            _sErrorMessage += "\nLast_useful_checkpoint:_\" +
                _pJavaParser.getLastFunction() + "\"\n";
        }
        _sErrorMessage += pParseException.getMessage() + "\n";
        _throwError = pParseException;
        throw pParseException;
    } catch (Error pTokenMgrError) {
        if (_sErrorMessage == null) {
            _sErrorMessage = "";
        }
        _sErrorMessage += "TokenMgrError_in_STDIN\n";
        _sErrorMessage += pTokenMgrError.getMessage() + "\n";
        _throwError = pTokenMgrError;
    }
    throw pTokenMgrError;
}

private void manageDebugger(Reader reader, boolean
    parser15) {
    if (Util.isDebugEnabled()) {
        if (parser15) {
            Util.debug("creating _JavaParser15Debug");
            _pJavaParser = new JavaParser15Debug(reader);
        } else {
            Util.debug("creating _JavaParserDebug");
            _pJavaParser = new JavaParserDebug(reader);
        }
    } else {
        if (parser15) {
            Util.debug("creating _JavaParser15");
            _pJavaParser = new JavaParser15(reader);
        } else {
            Util.debug("creating _JavaParser");
            _pJavaParser = new JavaParser(reader);
        }
    }
}

private void iteratePkgs() {
    Map htNewPackages = _pJavaParser.getPackage();
    for (Iterator ePackages = htNewPackages.entrySet().
        iterator(); ePackages.hasNext();) {
        String sPackage = (String) ((Map.Entry) ePackages.next()
            ).getKey();

        PackageMetric pckmNext = (PackageMetric) htNewPackages
            .get(sPackage);
        pckmNext.name = sPackage;

        PackageMetric pckmPrevious = (PackageMetric)
            _htPackages.get(sPackage);
        pckmNext.add(pckmPrevious);

        _htPackages.put(sPackage, pckmNext);
    }
}
```

Listing 2: Long Method example (refactored)